

Week-4Day-1

The screenshot shows a web browser window with the URL `D:/GenAIHandson/fine-tuning-test-data-builder.html`. The page title is "JSONL Training Data Converter for Fine tuning LLM" with a subtitle "LLM Fine-tuning Dataset Builder - 7 Frameworks". The interface is divided into two main sections: "INPUT" and "OUTPUT".

INPUT Section:

- 1 SELECT FRAMEWORK & LANGUAGE:** This section contains buttons for Selenium (Java, C#, Python) and Playwright (Java, TypeScript, C#, Python). Selenium buttons are green, and Playwright buttons are blue.
- 2 PROMPT CONFIGURATION:** A dropdown menu currently set to "DEFAULT".
- 3 ADD TRAINING DATA:** This section includes a "+ Single Entry" button and a "Batch Upload" button. A blue informational box states: "After clicking **Add Entry**, the entry is saved and a per-entry download button appears automatically in the **Downloads** tab on the right. Leave **Page Object Code** blank to auto-generate it from the HTML using the selected framework's template." Below this is a text area labeled "HTML DOM" with the placeholder "Paste your HTML DOM here..." and a "required" label.

OUTPUT Section:

- 4 Output:** The active tab, showing "0 entries in dataset" and a "Min 10 required for fine-tuning" warning. It includes input fields for "FRAMEWORK" and "LANGUAGE", both currently empty. A message below reads: "No framework selected -- choose one on the left".
- 5 Downloads:** A tab for viewing downloaded files.
- Format Ref:** A link to format reference.
- Live Preview (last 3 entries):** A text area showing "// Entries appear here as you add them...".
- Clear All Entries:** A red button at the bottom of the output section.

The bottom of the browser window shows a Windows taskbar with the search bar, taskbar icons, and system tray showing the time as 6:41 AM on 6/28/2026.

Recommended step-by-step order

1. Add the new `sel-rb` entry to `FW`.
2. Add the new prompt text to `SYS`.
3. Add the Ruby locator handling in `locStr(...)`.
4. Add the Ruby branch in `generateTemplate(...)`.
5. Add the Ruby format example in `FMT`.
6. Update the UI text and the format reference table.
7. Test with sample HTML to confirm:
 - the framework appears in the picker
 - selecting it updates the framework info
 - "Load Sample Template" generates Ruby code
 - batch processing uses `===RUBY===` correctly

Small UX improvement worth adding

To make the default template feel more immediate, you can also auto-generate the Ruby sample code when the framework is selected and the HTML textarea already has content.

For example, inside `pickFw(...)`, after setting `selFw`, you can check whether `htmlIn` already contains text and auto-fill `codeIn` using `generateTemplate(...)`.

That makes the experience closer to your requirement: "default templates for the new framework when selected."

🔗 ⓘ Create Finetuning Model.txt:1-26

Ask a question about your code or project

+ ⓘ Ask | Auto | ⚙️

Activate Windows



Go to Settings to activate Windows.

🖨 Local | 🛡 Default Approvals

Selenium Ruby added

The screenshot displays the 'JSONL Training Data Converter for Fine tuning LLM' web application. The interface is divided into two main sections: 'INPUT' and 'OUTPUT'.

INPUT Section:

- 1 SELECT FRAMEWORK & LANGUAGE:** This section shows a grid of framework and language combinations. 'Selenium' is selected for the framework, and 'Ruby' is selected for the language. Other options include Selenium (Java, C#, Python), Playwright (Java, TypeScript), and Playwright (C#, Python).
- 2 PROMPT CONFIGURATION:** Currently set to 'DEFAULT'.
- 3 ADD TRAINING DATA:** Includes a '+ Single Entry' button and a 'Batch Upload' button. A note states: 'After clicking Add Entry, the entry is saved and a per-entry download button appears automatically in the Downloads tab on the right. Leave Page Object Code blank to auto-generate it from the HTML using the selected framework's template.' Below this is a text area for 'HTML DOM' with a 'required' label and a 'Paste your HTML DOM here...' placeholder.

OUTPUT Section:

- 4 Output:** Shows '0 entries in dataset' and a 'Min 10 required' warning for fine-tuning. It includes fields for 'FRAMEWORK' and 'LANGUAGE', both currently empty, with a message 'No framework selected -- choose one on the left'. Below is a 'Live Preview (last 3 entries)' section with a placeholder '// Entries appear here as you add them...'.
- 5 Downloads:** This tab is active and shows a list of 'PER-ENTRY DOWNLOADS' for 10 entries. Each entry is labeled 'Selenium - Ruby' and has a 'Download' button. The first entry has a truncated text 'Lea"7L68...'. Below the list is a 'DOWNLOAD FULL DATASET' section with the filename 'training_data_selenium_ruby.jsonl' and a 'Download All as JSONL' button.
- Format Ref:** A tab for format reference.

The application is running in a browser window with the address bar showing 'D:\GenAI\Handson\fine-tuning-test-data-builder.html'. The Windows taskbar at the bottom shows the time as 6:57 AM on 6/28/2026, with a temperature of 28°C and a sunny weather forecast.

```
1  ent": "You are an expert in generating Selenium Ruby page object classes using provi
2  ent": "You are an expert in generating Selenium Ruby page object classes using provi
3  ent": "You are an expert in generating Selenium Ruby page object classes using provi
4  ent": "You are an expert in generating Selenium Ruby page object classes using provi
5  ent": "You are an expert in generating Selenium Ruby page object classes using provi
6  ent": "You are an expert in generating Selenium Ruby page object classes using provi
7  ent": "You are an expert in generating Selenium Ruby page object classes using provi
8  ent": "You are an expert in generating Selenium Ruby page object classes using provi
9  ent": "You are an expert in generating Selenium Ruby page object classes using provi
10 ent": "You are an expert in generating Selenium Ruby page object classes using provi
11
```

jsonl.co/jsonl-formatter

Viewer Tools Guide us English

Input JSONL

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are an expert in generating Selenium Ruby page object classes using provided DOM information. Use Selenium WebDriver Ruby conventions, proper locators, and insert TODO comments only when a mapping is missing. Make sure every page method includes comments in the generated code. Generate only code grounded in the supplied DOM."
    },
    {
      "role": "user",
      "content": "Given the following DOM structure:\n<html>\n<input class='input_login' type='text' id='username' name='USERNAME' size='50'/>\n<input class='input_login' type='password' id='password' name='PASSWORD' size='50'/>\n<input class='decorative_submit' type='submit' value='Login'/>\n\nGenerate a Selenium Ruby page object class for this DOM."
    },
    {
      "role": "assistant",
      "content": "require 'selenium-webdriver'\n\nclass GeneratedPage\n  def initialize(driver)\n    @driver = driver\n  end\n\n  def enter_username(username)\n    @driver.find_element(:id, 'username').send_keys(username)\n  end\n\n  def enter_password(password)\n    @driver.find_element(:id, 'password').send_keys(password)\n  end\n\n  def click_login()\n    @driver.find_element(:css, 'input[type='submit']').click\n  end\nend\n\nend"
    }
  ]
}
```

Options

Indent: 2 spaces 4 spaces Tab Compact (minify) Sort keys

1 lines 1 valid

Formatted Output

```
class GeneratedPage
  def initialize(driver)
    @driver = driver
  end

  def enter_username(username)
    @driver.find_element(:id, "username").send_keys(username)
  end

  def enter_password(password)
    @driver.find_element(:id, "password").send_keys(password)
  end

  def click_login()
    @driver.find_element(:css, "input[type='submit']").click
  end
end
end
}
```

Output: 1.3 KB

Activate Windows
Go to Settings to activate Windows.